

Reflection and Approach

This is a Natural Language Processing (NLP) project. In this project, I built a system that can automatically read and understand news articles.

The NewsBot Intelligence System can:

-  Preprocess news articles using advanced text cleaning techniques
-  Classify articles into categories (Politics, Sports, Technology, Business, Entertainment, Health)
-  Analyze sentiment and emotional tone of news content
-  Identify patterns using TF-IDF, POS tagging, and syntax parsing
-  Generate insights for media monitoring and business intelligence
-  Summarize news
-  Translate news and detect languages
-  Name entities

This project was built using:

- spaCy and NLTK: the main libraries for text processing and NLP tasks
- scikit-learn: used to train and evaluate the classification models
- BBC News dataset: a public dataset of 2,225 news articles, used to train and test the system
- Google Colab: the development environment used to write and run the code
- Langdetect
- Deep-translator
- Sumy

This project follows a modular development approach. Instead of writing all the code in one large file, each function was built and tested separately in its own notebook. Once each function was tested and working correctly, it was exported into its own Python file inside the 'newsbot/' folder. Finally, all these functions were imported together into one main notebook, where the complete NewsBot system was built and tested with real news articles.

Preprocessing Module

First, I installed all the libraries needed for this project, such as spaCy, NLTK, and pandas. Then, I mounted Google Drive. This is where the dataset is stored, and where every notebook needed to build the NewsBot is saved. Working in Drive means the files stay saved even after the Colab session ends. After mounting Drive, I loaded the dataset and checked that it was imported correctly using `head()`, `shape`, and `info()`. This confirmed that the data was loaded

properly before moving on. Next, I built the `clean_text()` function. These functions use regular expressions to remove things that add noise to the text, such as URLs, email addresses, and HTML tags. I tested this function on the first article in the dataset to check that it worked as expected. After that, I built the `preprocess_text()` function. This function uses `spaCy` to tokenize the text (split it into individual words) and lemmatize it (reduce each word to its base form). It also removes stop words and punctuation, since these do not add useful meaning for later analysis. I tested this function as well and confirmed that it returned a clean list of lemmas. Finally, I exported both functions into a single file, `preprocessing.py`, using `%%writefile`. This file will later be imported into the main NewsBot notebook, where the complete system is built.

TF-IDF Module

I start the same way as before: mounting Google Drive, loading the dataset, and checking that it loaded correctly. This first step is repeated in almost every notebook in this project, since each one runs in its own separate Colab session. Next, I import the functions I already built and tested in the previous notebook (`clean_text` and `preprocess_text`) and apply them to the dataset to get it ready for analysis.

Once the text is clean and processed, I build the TF-IDF vectorizer. I set `max_features` to 5000, which is a conservative number, enough to capture the most useful words without making the dataset too large to handle. I also use `ngram_range=(1,2)`, meaning I look at both single words (unigrams) and pairs of words (bigrams). This helps me keep more information about how groups of words work together, without growing the vocabulary too much. For `min_df`, I use 2. This means I ignore any word that appears in only one document, since it could be a typo or a very rare word that doesn't help me understand the data. For `max_df`, I use 0.8, which means I ignore words that appear in more than 80% of the documents. These words are so common that they may act like "hidden" stop words, adding no useful information.

After that, I run `fit_transform` on the processed text and get the feature names. With these, I build the final TF-IDF DataFrame.

Finally, I build the `get_top_tfidf_terms()` function. This function shows the top words (in this case, the top 10) that are most important for a given category. It takes three arguments: the TF-IDF DataFrame, the category I want to look at, and how many top words I want to see. I tested the function to confirm it works correctly and then exported it into a `.py` file, so it can be imported later in the main NewsBot notebook.

POS Analysis Module

For this notebook, I only needed to import `clean_text`, since I do not want to remove stop words here. Removing stop words would take away important information about how adverbs and other grammar elements are used, which is exactly what I want to analyze in this module.

For the `analyze_pos_patterns` function, I used `spaCy` and `Counter` to build a dictionary for each text, containing the proportion of each POS tag found in that text. I then tested the function by going through each row of the dataset, applying the function, and adding the category and article ID to the resulting dictionary. After that, I converted all the results into a `DataFrame`.

Finally, I exported the function into a `.py` file.

Syntax Parsing Module

I import `clean_text` again, since for this function I do not want to remove stop words, as I want to do semantic analysis, and stop words carry grammatical information I need.

To build the function, I use `spaCy` to process the text, then go through each token, skipping spaces and punctuation, and collect its dependency relation. I also separate out the subjects and objects of each sentence, and group full noun phrases, using `spaCy`'s built in noun chunk detection. At the end, I count how many times each dependency relation appears and return everything as a dictionary.

I tested the function with a very simple sentence, to check that it worked correctly, and, as I can see, the analysis is quite good. After that, I exported the function into a `.py` file.

Classifier Module

I start by loading the dataset and checking that it is loaded correctly. Then I import `train_test_split` to divide the dataset into a training set and a test set, using an 80/20 split. After that, I split the data into the usual structure (`x_train`, `y_train`, `x_test`, `y_test`), and check that the split worked correctly using `head()` and `shape()`.

Next, I encode `y_train` and `y_test` using `scikit-learn`'s `LabelEncoder`, which converts the category names into numbers the model can work with. I then import the functions built in the previous notebooks and apply them to `x_train` and `x_test`, to build the final dataset using both the TF-IDF analysis and the POS analysis.

After that, I ran cross-validation to choose the model that performed best. In this case, the best model was `LinearSVC`. I also ran a randomized search to try to improve it, but the results showed that the base model already performed better than the tuned versions. So, I trained the base model, and evaluated it, getting strong and consistent metrics, in line with what I saw during training. Finally, I saved the model using `joblib`, so it can later be used to build the NewsBot agent.

Sentiment Analysis Module

For this notebook, I use `NLTK` instead of `spaCy`, since `spaCy` does not have a built-in sentiment tool. I use `VADER`, which is `NLTK`'s sentiment analyzer, made specifically for this task. I did not apply `clean_text` here, since `VADER` works better with the original text. It uses

things like capital letters and punctuation as signals of emotional intensity, and cleaning the text would remove that information.

For the `analyze_sentiment` function, I pass the raw text into VADER's `polarity_scores()` method, which returns a dictionary with a compound score (an overall sentiment score from -1 to 1), and three more values: how positive, neutral, and negative the text is. Then, based on the compound score, I add a simple label: positive, negative, or neutral. I tested the function with three short example sentences, one clearly positive, one clearly negative, and one neutral, just to confirm the function behaves as expected. After confirming it worked correctly, I exported it into a `.py` file.

Named Entity Recognition (NER) Module

For this notebook, I only needed to import `clean_text`, since I do not need lemmatization or stop word removal for entity recognition. Removing words or changing their form could break the names, organizations, and locations that spaCy needs to detect as a single unit.

To build the function, I load spaCy's small English model (`en_core_web_sm`) and process the cleaned text with it. spaCy automatically tags spans of text as entities, so I go through `doc.ents` and collect each entity's text together with its label (for example, PERSON, ORG, or GPE). I return the result as a list of tuples, since this format is simple to read and simple to plug into the rest of the system later.

I tested the function on the first cleaned article of the dataset and confirmed that it correctly picked out people, organizations, and places mentioned in the text. After confirming it worked, I exported it into its own file, `ner.py`, following the same modular approach I used for every other function in this project.

Language Detection and Translation Module

This module actually combines two small, separate functions, since both technologies solve related but different problems: knowing what language a text is written in, and converting text from one language into another.

For language detection, I use the `langdetect` library. This library is a Python port of Google's language detection tool, and it only needs one line to work: I pass in the raw text and it returns a language code, such as 'en' for English or 'es' for Spanish. I did not need to clean or preprocess the text first, since the library is designed to work directly on raw text.

For translation, I use `deep_translator`, which gives access to Google Translate without needing an API key. The function takes the text and a target language code, and returns the translated text. I set the source language to 'auto' so the function does not need to know beforehand what language the input is written in.

I tested both functions together using three short sample sentences written in Spanish, English, and Japanese. For each sample, I printed the detected language and then the same sentence translated into English. All three samples were detected and translated correctly, which confirmed both functions were ready to be used inside the main agent. After testing, I exported them into two separate files, `language_detection.py` and `translation.py`, keeping each function isolated in its own module.

Summarization Module

For this notebook, I use the `sumy` library, which is built specifically for automatic text summarization and does not require training a model of my own. `Sumy` offers a few different summarization algorithms, and I chose the LSA (Latent Semantic Analysis) summarizer, since it looks at the relationships between sentences and terms across the whole text, rather than only ranking sentences by individual word frequency.

The function takes the raw article text, parses it into sentences using `sumy`'s tokenizer, and then asks the LSA summarizer to return the three most representative sentences. I loop through the result and collect each sentence as a string, so the final output is a short list of sentences that work as an extractive summary of the article.

I tested the function with a real news article about the passing of Lionel Messi's father, and it correctly picked out the three sentences that carried the most important information from the story, while leaving out smaller supporting details. After confirming the output made sense, I exported the function into `summarize.py`.

Intent Classification and Conversational Interface Module

This module is what turns the NewsBot from a set of separate functions into something the user can actually talk to. It is made up of three functions that work together in a chain: `classify_intent`, `process_query`, and `generate_response`.

The `classify_intent` function takes the user's raw text query and figures out what the user is asking for. Instead of training a machine learning model for this (which was out of scope given the time available), I used a rule-based approach: the function checks the query for keywords. If it finds the name of a news category (business, entertainment, politics, sport, tech), it stores that as the requested category. If it finds words like 'translate', 'summarize', 'entity', or 'language', it sets the requested action accordingly. If none of these keywords appear, the default action is to simply show an article. The function returns all of this as a small dictionary, which acts as a structured instruction for the next step.

The `process_query` function takes that dictionary and does the actual work. If the action is to show an article, it filters the dataset by the requested category (or picks from the whole dataset if no category was given), selects one article at random, and runs it through the sentiment analysis and POS tagging functions built in earlier modules. This analysis is stored in a variable called `current_article`, which stays available outside of the function. If the action

is instead to translate, summarize, extract entities, or detect the language, the function does not pick a new article. It reuses the text already stored in `current_article` and passes it into the corresponding function from the modules described above. This is what lets the user say something like 'show me a tech article' followed by 'translate it', without having to repeat or resend the article text.

The `generate_response` function takes whatever `process_query` returned and turns it into a readable message for the user. Depending on the action, it formats the response differently: for a new article, it shows the predicted category, the sentiment, the POS tags, and a preview of the text; for a translation, summary, entity list, or detected language, it formats that specific result into a short sentence. If no article had been shown yet and the user asks to translate or summarize, the function returns a message explaining that an article needs to be requested first, instead of letting the program crash.

I tested the three functions together with a handful of example queries before connecting them to the full agent, checking that filtering by category worked, that follow-up actions correctly reused the last article shown, and that the response text read naturally. Once I confirmed the chain worked end to end, I exported the three functions together into a single file, `intent.py`, since they are meant to always be used as a group.

NewsBot Agent Module (Updated for 2.0)

For the final version of the agent, I first load the news dataset and keep it to a sample of 300 articles, mainly to keep the prediction step fast during testing, since running the full classification pipeline on every article takes noticeably longer with a larger dataset.

Before starting the conversation loop, I run every article in the dataset through `predict_category_pipeline`, a function that repeats the same steps used to train the original classifier: cleaning and preprocessing the text, building its TF-IDF vector, extracting its POS features, lining the combined features up with the columns the model was trained on, and finally predicting the category with the saved LinearSVC model. The predicted category for each article is decoded back into its real name using the saved label encoder and stored in a new column, `Predicted_Category`. Doing this once, before the conversation starts, means the agent does not need to reclassify articles every time the user asks for a category, which keeps the conversation responsive.

With the dataset ready, I import all the functions built across the previous modules: preprocessing, syntactic feature extraction, POS analysis, sentiment analysis, named entity recognition, translation, language detection, summarization, and the intent, query, and response functions described above. I also load the trained classifier, TF-IDF vectorizer, label encoder, and feature column list saved with joblib during the midterm.

The `NewsBotAgent` function itself is a loop: it greets the user, then repeatedly asks for a query using `input()`. Each query is passed through `classify_intent`, then `process_query`, then

generate_response, and the resulting message is printed back to the user. The loop keeps running until the user types 'goodbye', at which point the agent prints a farewell message and stops. This gives the whole system a simple but functional conversational interface, letting the user request articles by category, ask for a new one, and apply translation, summarization, entity extraction, or language detection to whatever article is currently being discussed, all without writing any code themselves.

I tested the full agent interactively by typing a sequence of queries: asking for an article from a specific category, then asking to translate it, then asking to summarize it, then asking for a different article. The agent correctly kept track of which article was 'current' between messages, and only reclassified or refetched an article when explicitly asked to, matching the behavior I had originally planned for the system.